

The Agentic Loop in Claude Code

Components: Context, Tools, and Permissions — with worked examples

The agentic loop is the core execution cycle that lets Claude Code go from "read a request" to "verified working change" without a human manually shepherding every step. Each iteration follows roughly:

Perceive → **Plan** → **Act** → **Observe** → **Repeat** — until the task is done or a boundary requires human input.

Three components make this loop work together: **context**, **tools**, and **permissions**.

1. Context

Context is everything Claude "knows" at a given point in the loop: the conversation history, file contents it has read, system reminders, project instructions (like a `CLAUDE.md`), and tool results from earlier steps.

How it interacts with the loop:

- Each tool result gets folded back into context, which reshapes the next decision. Reading a file changes what "the right next action" looks like.
- Context accumulates across turns but is finite — long sessions get summarized so the loop can keep running without losing the thread.
- Project-level context (e.g., a `CLAUDE.md`) is loaded up front and persists across the whole session, acting as a standing constraint on every subsequent decision.

Example: In a session governed by a `CLAUDE.md`, before touching any code Claude reads the file, which establishes rules (e.g., "escalate only for strategic decisions," "update PROGRESS.md after every change"). That context then silently shapes every later step in the loop — Claude doesn't re-ask whether to write a test for new business logic, because context already answered that question.

2. Tools

Tools are the loop's only way to affect the world beyond generating text — reading/writing files, running shell commands, searching code, fetching URLs, spawning subagents, etc. Each tool call is a discrete, typed action with a defined input/output contract.

How it interacts with the loop:

- Claude selects a tool based on current context ("I need to find where `X` is defined" → `Grep`, not `Bash grep`).
- The tool's *result* becomes new context, which determines the next tool choice. This is the "Act → Observe" half of the cycle repeating.
- Tools can be parallelized when independent (e.g., reading three unrelated files at once) or sequenced when dependent (e.g., read a file before editing it).

Example: Fixing a bug might look like: `Grep` for the failing function name → `Read` the surrounding file → `Edit` the specific lines → run tests via a shell command → observe failure output → `Edit` again → rerun. Each tool call's output is the direct input to the next decision — that chain *is* the loop.

3. Permissions

Permissions are the gate between "Claude decided to act" and "the action actually happens." They determine whether a tool call executes automatically, prompts the user for approval, or is blocked outright.

How it interacts with the loop:

- Low-risk, reversible tool calls (reading a file, running a linter) typically execute without interruption, keeping the loop fast.
- Higher-risk or hard-to-reverse actions (force-push, `rm -rf`, sending an email, deploying) pause the loop for explicit user confirmation — even if Claude's plan says to proceed.
- Permissions can be configured per-project (`.claude/settings.json`) to pre-authorize certain safe patterns (e.g., allow `npm test` without prompting) so the loop doesn't stall on routine, low-blast-radius operations.

Example: In a governance-heavy repo, permissions and an "Autonomy Model" work together: implementation-level changes (renaming a variable, adding a test) proceed without asking, but anything crossing a governance boundary (schema redesign, new external dependency,

production infra change) is expected to pause and escalate — regardless of how confident Claude is. Permissions are the mechanical enforcement of that policy at the tool-call level.

Putting It Together

A single coding session is really many loop iterations chained together, with context accumulating, tools executing, and permissions gating the risky ones:

1. **Context:** Claude reads `CLAUDE.md` and the user's request.
2. **Plan:** Decides a bug fix requires finding a function definition.
3. **Tool:** Calls `Grep`, gets file/line results → new context.
4. **Tool:** Calls `Read` on that file → context now includes full function body.
5. **Tool:** Calls `Edit` to apply the fix — permission check passes (local, reversible) → executes automatically.
6. **Tool:** Runs the test suite via shell — result folds back into context.
7. **Permission boundary:** Claude wants to `git push` — permission mode requires explicit user confirmation, so the loop pauses here rather than completing autonomously.
8. **Loop ends** (or continues) based on the user's response.

The loop's power comes from this tight feedback cycle — each tool's real-world result immediately reshapes context, which reshapes the next action — while permissions ensure autonomy stays bounded to what's actually safe to do without a human in it.

Generated by Claude Code